Занятие 6: Основы автоматического тестирования

Unit, Integration, E2E тесты, Arrange-Act-Assert

Слайд 1: Уровни тестирования

Unit Tests (Модульные тесты)

- Тестируют отдельные функции/классы
- Быстрые
- Изолированные
- Моки зависимостей

Integration Tests (Интеграционные тесты)

- Тестируют взаимодействие компонентов
- Средняя скорость
- Реальные зависимости (частично)

End-to-End Tests (E2E тесты)

- Тестируют весь поток
- Медленные
- Реальное окружение
- Браузер, БД, API

Слайд 2: Тестирование API

Что тестируем:

- Endpoints
- Статус-коды
- Структура ответов
- Валидация
- Бизнес-логика

Границы проверяемой системы:

- Unit: отдельный handler/service
- Integration: handler + service + repository
- E2E: полный HTTP запрос с реальной БД

Слайд 3: Arrange-Act-Assert

Шаблон написания теста:

Arrange (Подготовка)

- Создание тестовых данных
- Настройка моков
- Инициализация окружения

Act (Действие)

- Вызов тестируемого кода
- Выполнение операции

Assert (Проверка)

- Проверка результата
- Сравнение с ожиданием
- Проверка побочных эффектов

Слайд 4: Пример Arrange-Act-Assert

```
def test_create_user():
    # Arrange
    user_data = {"name": "John", "email": "john@example.com"}
    mock_repo = Mock()
    mock_repo.email_exists.return_value = False
    service = UserService(mock_repo)

    # Act
    user = service.create_user(user_data['name'], user_data['email'])

    # Assert
    assert user.name == "John"
    assert user.email == "john@example.com"
    mock_repo.save.assert_called_once()
```

Слайд 5: Обычные сценарии

Happy Path - всё работает корректно

```
def test_create_user_success():
    # Arrange
    user_data = {"name": "John", "email": "john@example.com"}
    mock_repo = Mock()
    mock_repo.email_exists.return_value = False
    service = UserService(mock_repo)

    # Act
    user = service.create_user("John", "john@example.com")

    # Assert
    assert user.name == "John"
    assert user.email == "john@example.com"
```

Слайд 6: Граничные сценарии

Edge Cases - граничные значения

```python
def test_create_user_min_age():
    # Arrange
    service = UserService(mock_repo)

    # Act
    user = service.create_user("John", "john@example.com", age=18)

    # Assert
    assert user.age == 18

def test_create_user_max_age():
    # Arrange
    service = UserService(mock_repo)

    # Act
    user = service.create_user("John", "john@example.com", age=120)

    # Assert
    assert user.age == 120

def test_create_user_empty_name():
    # Arrange
    service = UserService(mock_repo)

    # Act & Assert
    with pytest.raises(ValueError):
        service.create_user("", "john@example.com")
```

Слайд 7: Ошибочные сценарии

Error Cases - обработка ошибок

```python
def test_create_user_duplicate_email():
    # Arrange
    mock_repo = Mock()
    mock_repo.email_exists.return_value = True
    service = UserService(mock_repo)

    # Act & Assert
    with pytest.raises(ValueError, match="Email already exists"):
        service.create_user("John", "john@example.com")

def test_create_user_invalid_email():
    # Arrange
    service = UserService(mock_repo)

    # Act & Assert
    with pytest.raises(ValueError, match="Invalid email"):
        service.create_user("John", "invalid-email")
```

Слайд 8: Регрессионный тест

Регрессия - повторное появление ошибки

```python
def test_bug_fix_price_calculation():
    """
    Тест для исправления бага:
    неправильный расчёт цены при скидке
    """
    # Arrange
    product = Product(price=100, discount=10)

    # Act
    final_price = product.calculate_final_price()

    # Assert
    assert final_price == 90  # 100 - 10% = 90
    # Баг был: возвращал 10 вместо 90
```

Слайд 9: Тестовые подмены зависимостей

Mock - имитация зависимости

```
from unittest.mock import Mock

# Создание мока
mock_repo = Mock()
mock_repo.find_by_id.return_value = User(id=1, name="John")

# Использование
user = mock_repo.find_by_id(1)
assert user.name == "John"

# Проверка вызова
mock_repo.find_by_id.assert_called_with(1)
```

Stub - заглушка

```
def stub_find_by_id(user_id):
    return User(id=user_id, name="Stub User")
```

Слайд 10: Ограничения mocks

Что можно мокать:

- Внешние API
- База данных
- Файловая система
- Время
- Случайные значения

Что НЕ стоит мокать:

- Тестируемый код
- Простые объекты (DTO)
- Value objects

Проблемы с моками:

- Слишком много моков = хрупкий тест
- Моки могут не соответствовать реальности
- Тестируем моки, а не код

Слайд 11: Покрытие кода тестами

Что измеряет coverage:

- Какой процент кода выполнен при тестах
- Какие строки/ветки/функции покрыты

Что НЕ доказывает:

- Корректность кода
- Отсутствие багов
- Качество тестов

Пример:

```
def divide(a, b):
    return a / b

# Тест с 100% coverage, но не проверяет b=0
def test_divide():
    assert divide(10, 2) == 5  # coverage 100%
```

Слайд 12: Пример unit-теста

```
def test_user_service_create_user():
    # Arrange
    mock_repo = Mock()
    mock_repo.email_exists.return_value = False
    mock_repo.save.return_value = User(id=1, name="John", email="john@example.com")
    mock_email = Mock()
    service = UserService(mock_repo, mock_email)

    # Act
    user = service.create_user("John", "john@example.com")

    # Assert
    assert user.id == 1
    assert user.name == "John"
    assert user.email == "john@example.com"
    mock_repo.email_exists.assert_called_once_with("john@example.com")
    mock_repo.save.assert_called_once()
    mock_email.send_welcome.assert_called_once_with("john@example.com")
```

Слайд 13: Пример integration-теста

```
def test_create_user_integration():
    # Arrange
    db = TestDatabase()
    db.setup()
    repo = UserRepository(db)
    service = UserService(repo, Mock())

    # Act
    user = service.create_user("John", "john@example.com")

    # Assert
    assert user.id is not None
    assert user.name == "John"

    # Проверка в БД
    saved_user = repo.find_by_id(user.id)
    assert saved_user is not None

    # Cleanup
    db.cleanup()
```

Слайд 14: Практическое задание

Задание 1: Unit-тест
Напишите unit-тест для функции валидации email.

Задание 2: Integration-тест
Напишите integration-тест для создания пользователя.

Задание 3: Тестирование сценариев
Напишите тесты для обычного, граничного и ошибочного сценариев.

Слайд 15: Вопрос для собеседования

Вопрос: Выберите уровень теста и проверки результата для заданного требования.

Требование: "При создании пользователя с существующим email должен возвращаться ошибка"

Ожидаемый ответ:

- Уровень: Unit или Integration
- Arrange: создать мок репозитория, вернуть True для email_exists
- Act: вызвать create_user с существующим email
- Assert: проверить, что выбрасывается исключение с сообщением об ошибке